



AD1 – Projeto e Desenvolvimento de Algoritmos – 1º semestre/2026

Código da disciplina EAD05005

Nome: Gabriel Babo Casanova

Matrícula: 26113050056

Polo: Rocinha

Observações:

Para a resolução desta prova considere a existência em PETEQS das seguintes funções:

função tamanho(entradas: texto)

Retorna o número de caracteres na string de entrada.

Exemplo:

```
imprima tamanho("@123umaString")  
// imprimiria 13
```

função tamanho(entradas: vetor[])

Retorna o número de elementos no vetor de entrada.

Exemplo:

```
imprima tamanho([10, 20, 30, 40, 60])  
// imprimiria 5
```

Considere que o comando **leia** em PETEQS pode ler uma cadeia de caracteres e os armazenar em um vetor do tamanho da cadeia. Considere também que o comando **imprima** pode imprimir uma cadeia de caracteres. Como exemplo, suponha que na execução do trecho de algoritmo mostrado abaixo o usuário digitou a cadeia 'PETEQS'.

Exemplo:

```
imprima 'Entre com uma cadeia de caracteres'  
leia frase  
imprima frase  
imprima frase[3]  
// imprimiria PETEQS e depois T
```

1a questão (valor 5.0)

Você foi contratado por uma empresa de robótica. A sua tarefa é escrever um programa que leia um conjunto de comandos para um robô e o movimento em mundo plano de duas dimensões. As duas coordenadas (**px**, **py**) que definem a posição atual do robô são sempre números inteiros que podem ser negativos, positivos ou zero. Os comandos que o robô recebe são definidos por caracteres alfabéticos e são os seguintes:

- **N ou n**: mover-se uma casa para o norte, ou seja, aumentar em um a coordenada **py**;
- **S ou s**: mover-se uma casa para o sul, ou seja, diminuir em um a coordenada **py**.
- **L ou l**: mover-se uma casa para o leste, ou seja, aumentar em um a coordenada **px**.
- **O ou o**: mover-se uma casa para o oeste, ou seja, diminuir em um a coordenada **px**.
- **R ou r**: zerar as duas coordenadas do robô.
- **I ou i**: imprimir as coordenadas atuais do robô.

Estes comandos são armazenados em um vetor de caracteres fornecido pelo usuário. O programa deve testar cada um dos caracteres e executar o comando correspondente. Um caractere diferente dos mostrados acima é considerado um comando inválido. Neste caso o programa deve imprimir uma mensagem de erro, imprimir as coordenadas atuais do robô e parar a execução do programa. O robô começa sempre na posição $px = 0$, $py = 0$. Ao terminar o programa deve imprimir as coordenadas de onde o robô parou.

Solução:

```
programa moverrobo
inicio
    roboandando <- verdadeiro

    px <- 0
    py <- 0

    i <- 1

    imprima 'Insira a sequência de comandos para o robô:'
    leia comandos
    tam <- tamanho(comandos)

    enquanto (i <= tam) E (roboandando = verdadeiro) faça
        comandoatual <- comandos[i]

        se (comandoatual = 'I') OU (comandoatual = 'i') então
            imprima px, py
        senão
            se (comandoatual = 'R') OU (comandoatual = 'r') então
                px <- 0
                py <- 0
                imprima 'Coordenadas zeradas'
            senão
                se (comandoatual = 'N') OU (comandoatual = 'n') então
                    py <- py+1
                senão
                    se (comandoatual = 'S') OU (comandoatual = 's') então
                        py <- py-1
                    senão
                        se (comandoatual = 'L') OU (comandoatual = 'l') então
                            px <- px+1
                        senão
                            se (comandoatual = 'O') OU (comandoatual = 'o') então
                                px <- px-1
```

```

                                senão
                                imprima 'Caractere inválido!'
                                roboandando <- falso
                                fim se
                            fim se
                        fim se
                    fim se
                fim se
            fim se
        fim se

        i <- i + 1
    fim enquanto

    imprima 'Coordenadas do robô:'
    imprima px, py
fim

```

2a questão (valor 5.0)

O número 6174 (seis mil, cento e setenta e quatro) é um número inteiro conhecido entre os matemáticos por suas propriedades. Por exemplo 6174 é divisível pela soma dos seus dígitos 18 ($6 + 1 + 7 + 4$);

O matemático indiano D. R. Kaprekar divulgou uma outra característica que fez o número 6174 ficar conhecido como constante de Kaprekar. Esta característica funciona da seguinte maneira:

1. Tome qualquer número de 4 dígitos, usando ao menos 2 dígitos diferentes (zeros complementares iniciais são permitidos).
2. Ordene os dígitos em ordem ascendente e depois em ordem decrescente, de modo a obter dois números a quatro dígitos, adicionando zeros à esquerda se necessário.
3. Subtraia o menor número do maior.
4. Repita o passo 2.

O processo acima, conhecido como **rotina de Kaprekar**, sempre convergirá para o valor 6174, em no máximo sete iterações. Assim que 6174 for alcançado, o processo continua a resultar no valor 6174 pois $7641 - 1467 = 6174$. Por exemplo, aplique a rotina no número 3524:

Iteração 1: $5432 - 2345 = 3087$

Iteração 2: $8730 - 0378 = 8352$

Iteração 3: $8532 - 2358 = \mathbf{6174}$

Escreva um programa em PETEQS que receba um número inteiro de quatro dígitos e imprima em quantos passos a rotina chegou ao número 6174 e os valores intermediários calculados pela rotina.

Solução:

```

programa kaprekar
inicio
    constante_kaprekar <- 6174
    passos <- 0
    numero_resultante <- 0
    numero_maior <- 0
    numero_menor <- 0

    imprima 'Digite um valor de até 4 dígitos que não sejam todos iguais'
    leia numero

```

```

enquanto (numero < 0) OU (numero > 9999) OU (numero = 0) OU (numero mod 1111 = 0) faça
    imprima 'Número inválido'
    imprima 'Digite um valor de até 4 dígitos que não sejam todos iguais'
    leia numero
fim enquanto

enquanto (numero <> constante_kaprekar) faça
    digitos[1] <- numero div 1000
    temp <- numero mod 1000

    digitos[2] <- temp div 100
    temp <- temp mod 100

    digitos[3] <- temp div 10
    digitos[4] <- temp mod 10

    para i de 1 até 3 faça
        para j de (i + 1) até 4 faça
            se (digitos[i] < digitos[j]) então
                aux <- digitos[i]
                digitos[i] <- digitos[j]
                digitos[j] <- aux
            fim se
        fim para
    fim para

    numero_maior <- (digitos[1]*1000) + (digitos[2]*100) + (digitos[3]*10) + digitos[4]
    numero_menor <- (digitos[4]*1000) + (digitos[3]*100) + (digitos[2]*10) + digitos[1]

    numero_resultante <- numero_maior - numero_menor
    numero <- numero_resultante
    passos <- passos + 1

    imprima 'Iteração: ',passos,': ',numero_maior,' - ',numero_menor,' = ',numero_resultante
fim enquanto

imprima 'Total de passos até chegar ao 6174: ', passos
fim

```